

BUS BOOKING SYSTEM FOR COLLEGE HOSTEL TO TOWN WEEKEND TRANSPORT MANAGEMENT

CLOUD COMPUTING

Submitted by

SAM DAVID (230701280)

SANJAY R (230701291)

SOORYA M P (230701325)

in partial fulfilment for the award of the degree of

**BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
MAY 2026
ANNA UNIVERSITY, CHENNAI**

BONAFIDE CERTIFICATE

Certified that this project report titled "BUS BOOKING SYSTEM FOR COLLEGE HOSTEL TO TOWN WEEKEND TRANSPORT MANAGEMENT" is the bonafide work of SAM DAVID, SOORYA M P, and SANJAY R who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy

Professor & Head of The Department
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr. S. PONMANI

Supervisor
Associate Professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. MALATHY E.M, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, DR. M. AYYADURAI M.E., Ph.D., Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

SAM DAVID

SANJAY R

SOORYA M P

ABSTRACT

Many students who live in the college hostel during weekdays return to their hometowns on weekends, but long-distance routes and travel coordination can be difficult to manage manually. This creates demand for a simple, verified transport booking workflow where students can reserve seats in advance, confirm their booking with OTP verification, and plan outbound and return trips with predictable timing. The system helps the transport team manage demand, seating, and passenger lists in a controlled manner.

The project aims to build a secure college transport booking platform where hostel students can reserve weekend travel seats and the transport office can manage route allocations efficiently. By restricting access to verified institutional credentials, the platform ensures that only eligible students can book trips. The primary focus is on long-distance route booking, stop selection, seat availability, and booking confirmation for outbound and return travel.

Built using a Django REST Framework backend hosted on Azure App Service, containerised with Docker and stored in Azure Container Registry, the system integrates with Azure Database for PostgreSQL as its production data store. A CI/CD pipeline automates image builds and deployments. The expected outcome is a reliable transport coordination platform that reduces manual booking effort, improves weekend travel planning, and helps the college manage hostel-to-town demand more efficiently.

TABLE OF CONTENTS

Chapter 1 Introduction	1
1.1 Problem Statement	2
1.2 Objective of the Project	4
1.3 Scope and Boundaries	6
1.4 Stakeholders and End Users	8
1.5 Technologies Used	10
1.6 Organization of the Report	12
Chapter 2 System Design and Architecture	14
2.1 Requirement Summary (Functional & Non-Functional)	15
2.2 Proposed Solution Overview	17
2.3 Cloud Deployment Strategy	19
2.4 Infrastructure Requirements	21
2.5 Azure Services Mapping and Justification	23
Chapter 3 DevOps Implementation	26
3.1 CI/CD Setup (GitHub Actions)	27
3.2 Containerization Strategy (Docker + ACR)	29
3.3 Infrastructure and Configuration	31
Chapter 4 Cloud Operations and Security	34
4.1 DevSecOps Integration	35
4.2 Monitoring and Observability	37
4.3 Access Control (RBAC Overview)	39
4.4 Deployment and Disaster Recovery Planning	41
Chapter 5 Results and Discussion	44
5.1 Implementation Summary	45
5.2 Challenges Faced and Resolutions	47
5.3 Performance and Cost Observations	49
5.4 Key Learnings and Team Contributions	51
Chapter 6 Conclusion and Future Enhancement	54
6.1 Conclusion	55
6.2 Future Scope	57
References	59
Appendices	61

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Students who stay in the hostel during weekdays need a reliable system to reserve long-distance bus seats for weekend trips back to their towns. Manual coordination through phone calls or informal messages creates several operational problems in terms of scalability, safety, and clarity:

- **Manual Booking Workload:** Transport staff must track seat demand, route preferences, and passenger lists without a centralized booking system. This increases the chance of overbooking or missed reservations.
- **Weekend Travel Peaks:** Demand increases sharply on Fridays and Sundays when hostel students return to their hometowns, creating the need for controlled seat allocation.
- **Verification and Safety:** Only authenticated college students should be allowed to book seats, ensuring the transport system remains restricted to the institution.
- **Route and Stop Management:** The college must coordinate buses, stops, departure dates, and pickup points in a way that is easy to manage and audit.
- **Convenience and Accessibility:** Students need a simple web-based workflow to check bus availability, select a stop, and confirm travel without visiting the transport office in person.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and implement a college-exclusive bus booking system that facilitates secure, efficient, and organised weekend transport reservations for hostel students.

1. **Enable Secure Seat Reservations:** Allow verified students to reserve bus seats for outward and return journeys using a simple authenticated workflow.
2. **Reduce Manual Transport Coordination:** Replace spreadsheets and phone-based coordination with a structured web application for booking and list management.
3. **Support Weekend Travel Planning:** Help hostel students plan routine weekend travel back to their towns with predictable route availability and stop selection.
4. **Deliver a Reliable, Scalable Cloud Platform:** Deploy the application on Microsoft Azure using App Service, Azure Container Registry, and Azure PostgreSQL Flexible Server, supported by a complete CI/CD pipeline via GitHub Actions.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the complete development and deployment of a full-stack web application with integrated cloud infrastructure. This includes student authentication, bus browsing, seat reservation workflows, OTP verification, cancellation, and transport administration.

The system supports core transport functionalities such as user registration scoped to a specific college, bus listing, route filtering, stop selection, and booking workflows. The backend

is containerized using Docker, stored in Azure Container Registry, and deployed on Azure App Service. Data persistence is handled by Azure Database for PostgreSQL Flexible Server.

A complete CI/CD pipeline is implemented using GitHub Actions to automate the process of building the Docker image, pushing it to the registry, and restarting the App Service deployment.

1.4 STAKEHOLDERS AND END USERS

Stakeholders: Project Developers / Team Members — Design, build, and maintain platform modules such as authentication, bus listing, booking logic, and admin exports.

End Users:

- Hostel Students who need to book buses for weekend travel to their hometowns.
- Transport Administrators who manage routes, bus capacity, and booking windows.
- Faculty or Staff responsible for verifying passenger lists and transport records.

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Layer	Technology	Purpose
Frontend	React + TypeScript	Building the student booking interface
Frontend	Tailwind CSS	Styling and UI components
Backend	Django (Python)	REST API, authentication, business logic
Backend	Gunicorn	WSGI HTTP server for Django in production
Backend	Django REST Framework	API serialization and view handling
Database	PostgreSQL 16	Relational database for structured transport data
Containerization	Docker	Packaging backend application into images
Registry	Azure Container Registry (ACR)	Storing and distributing container images
Hosting	Azure App Service	Running the Django backend container
Database Cloud	Azure PostgreSQL Flexible Server	Managed production database
CI/CD	GitHub Actions	Automating build, push, and deployment workflows

1.6 ORGANIZATION OF THE REPORT

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies, and report structure.

Chapter 2 presents the system design and architecture, including requirements, solution overview, cloud strategy, and service mappings.

Chapter 3 covers the DevOps implementation, including CI/CD pipelines, Docker containerization, and ACR integration.

Chapter 4 focuses on operational excellence and security, discussing DevSecOps practices, monitoring, access control, and deployment strategies.

Chapter 5 presents the results, including implementation summaries, challenges, performance metrics, and cost analysis.

Chapter 6 concludes with a summary of achievements and a roadmap for future enhancements.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The system consists of multiple cloud-based components deployed on Microsoft Azure to support a scalable, secure, and efficient bus booking platform.

The frontend of the application is developed using React and TypeScript. The backend logic is built with Django REST Framework, packaged as a Docker image, and hosted on Azure App Service (busbookingapi). The backend handles core functionalities such as login, bus listing, booking creation, OTP verification, and cancellation workflows.

Student data, bus information, stop records, and booking metadata are stored in a relational database provisioned via Azure Database for PostgreSQL Flexible Server (busbookingpg2026, version 16). The database name is bus_booking, accessed over SSL.

Container images are built and pushed to Azure Container Registry (busbookingcc.azurecr.io). The App Service pulls the latest image from ACR. Continuous integration and deployment pipelines are automated using GitHub Actions integrated with Azure credentials.

Non-Functional Requirements

- **Performance:** API response times below 300ms; page load times under 3 seconds, supported through Azure App Service and static frontend hosting.
- **Scalability:** The solution supports 500+ concurrent users via Azure App Service auto-scaling.
- **Availability:** The system is built on Azure's SLA-backed infrastructure to ensure 99.9% uptime.
- **Security:** College domain verification, secure user sessions, and database-level validation mechanisms ensure user identity and booking data remain protected.
- **Data Durability:** PostgreSQL geo-backup provides resilience against data loss events.
- **Cost Efficiency:** By leveraging the Azure Student Subscription, the project targets operation within ₹5,000 per month.

2.2 PROPOSED SOLUTION OVERVIEW

The proposed solution is a cloud-native platform that separates concerns across multiple layers. At the presentation layer, a React application delivers the user experience. This frontend communicates with a Django-based RESTful API backend hosted on Azure App Service.

The backend orchestrates all business logic, including student management, route selection, seat reservation, and booking status workflows. Data persistence is handled by Azure Database for PostgreSQL Flexible Server, which stores structured information about users, buses, stops, and bookings.

The entire container lifecycle is managed through Docker and Azure Container Registry. GitHub Actions automates the build, tag, push, and deploy process, ensuring that changes are validated before reaching production.

2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy leverages Microsoft Azure as the primary cloud provider, utilizing App Service for containerized backend hosting. The chosen region is Central India, providing low latency for the target user base.

The Django backend runs in Azure App Service (busbookingapi), with WEBSITES_PORT set to 8000 and DJANGO_SETTINGS_MODULE pointing to production settings. Azure Container Registry (busbookingcc) stores versioned backend images. Azure PostgreSQL Flexible Server serves as the relational database with SSL-enforced connectivity.

The overall request flow begins with the user interacting with the frontend. API requests are routed to the App Service backend, which interacts with PostgreSQL for data operations. This layered approach ensures clear separation of concerns and facilitates independent scaling.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources

- Azure App Service (busbookingapi): Runs the Django backend container from ACR. Auto-scaling based on load; WEBSITES_PORT=8000.
- Azure App Service (busbooking): Additional web app in the subscription for related deployments.

Storage and Database

- Azure PostgreSQL Flexible Server (busbookingpg2026): Version 16, located in Central India. Database name: bus_booking. Connectivity via sslmode=require. Fully qualified domain: busbookingpg2026.postgres.database.azure.com.
- Azure Container Registry (busbookingcc): Standard SKU. Login server: busbookingcc.azurecr.io. Stores backend images tagged as latest and versioned builds.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Requirement	Azure Service	SKU/Tier	Purpose	Est. Monthly Cost (₹)
Backend Hosting	Azure App Service (busbookingapi)	B1/Standard	Run Django backend container	Free (Student Credit)
Container Registry	Azure Container Registry (busbookingcc)	Standard	Store and distribute Docker images	~₹400

Requirement	Azure Service	SKU/Tier	Purpose	Est. Monthly Cost (₹)
Database	Azure PostgreSQL Flexible Server (busbookingpg2026)	Burstable B1ms	Production relational database	~₹1,200
CI/CD Automation	GitHub Actions	Free tier	Automated builds and deployments	Free
Additional Web App	Azure App Service (busbooking)	F1/Free	Supplementary app deployment	Free (Student Credit)

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CI/CD SETUP (GITHUB ACTIONS)

The project uses a CI/CD pipeline built with GitHub Actions to automate the container build and deployment process. The pipeline is triggered on every push to the main branch and follows these stages:

5. **Build and Push Docker Image:** The backend Django application is built into a Docker image using the Dockerfile in the repository. The image is tagged with both latest and a versioned tag (e.g., vYYYYMMDD-<shortsha>), then pushed to Azure Container Registry (busbookingcc.azurecr.io).
6. **Deploy to Azure App Service:** After the image is successfully pushed, the pipeline restarts the Azure App Service (busbookingapi) to pull and run the new container image. Secure access to Azure is handled through a Service Principal using credentials stored as GitHub secrets.
7. **Schema Migrations:** Django migrations are executed against the PostgreSQL database (DATABASE_URL) as part of the deployment. This ensures that schema changes are applied before traffic is routed to the new container.

Key GitHub secrets required for the pipeline:

- AZURE_CREDENTIALS – Service principal JSON for Azure authentication
- ACR_LOGIN_SERVER – busbookingcc.azurecr.io
- ACR_USERNAME and ACR_PASSWORD – ACR admin credentials
- DATABASE_URL – PostgreSQL connection string with sslmode=require
- DJANGO_SECRET_KEY – Django application secret key

3.2 CONTAINERIZATION STRATEGY (DOCKER + ACR)

The project uses a disciplined containerization strategy to package the Django backend into efficient, reproducible Docker images. The multi-stage build approach is used to minimize the final image size:

Image Build Process

- **Builder Stage:** Starts from the official Python image. Dependencies listed in requirements.txt are installed. Static files are collected using python manage.py collectstatic.
- **Runtime Stage:** The application code, dependencies, and collected static files are copied into the final image. Gunicorn is set as the container entry point listening on port 8000 (WEBSITES_PORT).

Deployment and Versioning

- The Azure Container Registry (busbookingcc) stores images at busbookingcc.azurecr.io/bus-booking-backend:latest.
- Versioned tags (e.g., vYYYYMMDD-<shortsha>) are used for production deployments, enabling precise tracking and quick rollbacks.

- Azure App Service is configured with WEBSITES_PORT=8000, DJANGO_SETTINGS_MODULE=transport_booking.settings_production, ALLOWED_HOSTS=*, and DATABASE_URL for the PostgreSQL connection.

3.3 INFRASTRUCTURE AND CONFIGURATION

All Azure resources are organized across two resource groups:

Resource Group	Resources	Purpose
busbooking-rg	App Service (busbookingapi), ACR (busbookingcc), PostgreSQL (busbookingpg2026)	Primary backend deployment resources
busbooking_group	App Service (busbooking)	Additional web application deployment

Key backend App Service settings:

- DJANGO_SETTINGS_MODULE=transport_booking.settings_production
- DATABASE_URL=postgresql://.../bus_booking?sslmode=require
- ALLOWED_HOSTS=* (to avoid Azure internal probe host issues)
- WEBSITES_PORT=8000

Database connectivity is secured via SSL (sslmode=require). No credentials are hardcoded; all sensitive values are injected via App Service environment variables sourced from GitHub Actions secrets during deployment.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 DEVSECOPS INTEGRATION

Security is embedded across the development lifecycle through lightweight, enforceable checks. Static application security testing is performed via language-native linters on both services: ESLint for the React frontend and Python linters (flake8, bandit) for the Django backend. These tools analyze source code for correctness, unsafe patterns, and security hygiene. Linting runs locally and in continuous integration on every pull request, and merges are blocked when violations are detected.

Container security is strengthened through minimal base images and deterministic builds. The backend is packaged using a Docker multi-stage build that reduces attack surface and minimizes potential CVEs. Images are built for the correct target platform (linux/amd64) to ensure consistent, reproducible deployments.

Operational consistency is maintained through environment-driven settings. Sensitive values such as `DATABASE_URL` and `DJANGO_SECRET_KEY` are injected via App Service environment variables rather than hardcoded. Database connectivity enforces SSL (`sslmode=require`). CORS is constrained to the exact frontend domain, aligning with the principle of least privilege.

4.2 MONITORING AND OBSERVABILITY

Monitoring and observability are achieved through Azure Monitor integrated with the App Service and PostgreSQL resources. Key performance indicators tracked include HTTP request counts, response times, CPU and memory utilization, and database query performance.

Alerting rules are configured in Azure Monitor to notify the team via email when critical thresholds are breached. Examples include CPU utilization exceeding 85% for five consecutive minutes or application restart frequency indicating instability. These alerts enable rapid incident response.

Application logs from the Django backend are streamed via Azure App Service diagnostic logging. Log retention policies ensure data is available for analysis. Azure Monitor's log queries help detect patterns such as unusual traffic spikes or repeated HTTP 500 errors.

4.3 ACCESS CONTROL

Access control is implemented at both infrastructure and application levels using least privilege principles.

At the Azure resource level, role-based access control (RBAC) grants the CI/CD Service Principal Contributor rights scoped to the target resource groups (`busbooking-rg` and `busbooking_group`), while developers have Reader access for troubleshooting.

Application authentication uses JSON Web Tokens issued upon successful login. Tokens include user identity claims and are validated by middleware on every request. Write operations require a valid token, while selected read endpoints allow optional authentication.

Database access is secured via App Service environment variables (DATABASE_URL, DB_PASSWORD). Connections enforce SSL (sslmode=require). Network exposure is minimized through Azure App Service ingress for the API and PostgreSQL firewall rules permitting Azure services access only.

4.4 DEPLOYMENT AND DISASTER RECOVERY PLANNING

Deployment strategy uses Azure App Service with rolling container updates. Each release builds a new container image via CI, pushes it to ACR, and restarts the App Service to load the latest image. App Service health checks ensure the new version is running correctly before traffic is fully served.

Rollback is performed by redeploying a prior image tag from ACR and restarting the App Service. Versioned images retained in the registry enable fast rollback while preserving a stable known-good state.

Disaster recovery leverages immutable container artifacts and infrastructure configuration for rapid restoration. Application images are stored and versioned in ACR. Sensitive configuration (database credentials, Azure identities) is sourced from GitHub Actions secrets to simplify secure re-provisioning. The managed PostgreSQL Flexible Server provides platform backups and point-in-time restore capabilities for data resilience.

Recovery time objectives target four hours for full environment re-provisioning and application restoration from ACR. Recovery point objectives target up to twenty-four hours of data loss under worst-case scenarios, aligned with managed database backup capabilities.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The bus booking platform was successfully implemented and deployed to Microsoft Azure, fulfilling all core objectives outlined in the project scope. The full-stack application provides a complete, secure user experience, including institutional authentication, bus listing management, booking confirmation, OTP verification, and transport coordination support.

The containerization strategy, leveraging multi-stage Docker builds, successfully produced a compact Django backend image. The GitHub Actions CI/CD pipeline automated the entire build-and-deploy process, achieving end-to-end deployment to Azure App Service in under 10 minutes. Database migrations were applied automatically during deployment, ensuring schema consistency.

Performance testing confirmed the efficiency of the Django backend, with core API endpoints demonstrating average response times well under 300ms under normal load. Database queries against Azure PostgreSQL Flexible Server consistently executed in sub-75ms times for standard read/write operations.

5.2 CHALLENGES FACED AND RESOLUTIONS

One of the main challenges faced was the App Service startup delay after container updates. The application took longer to respond after a new image deployment. To solve this, always-on settings were configured and health endpoint probes were added to ensure the container is warmed up before traffic is accepted.

The project also faced issues with ALLOWED_HOSTS misconfiguration. Azure App Service's internal load balancer uses a host header that did not match the application's configured domain, causing 400 Bad Request errors. The resolution was to set ALLOWED_HOSTS=* in the App Service configuration, which resolved internal probe failures.

A major bottleneck occurred with database SSL configuration. Initial connection attempts without sslmode=require were rejected by the PostgreSQL Flexible Server. The solution was to append ?sslmode=require to the DATABASE_URL connection string, enforcing encrypted connections.

Finally, the Docker image initially used a development-only Django server (runserver). Replacing this with Gunicorn as the WSGI server significantly improved throughput and stability in the production environment.

5.3 PERFORMANCE AND COST OBSERVATIONS

Performance Benchmarking

The Django backend demonstrates fast API response times under 300ms for most endpoints. Database operations against Azure PostgreSQL Flexible Server are highly responsive, with 95% of typical queries completing in under 75ms. The React frontend, served via CDN-backed static hosting, achieves excellent load times.

Metric	Target	Achieved
API Response Time	< 300ms	< 250ms (average)
Page Load Time	< 3 seconds	~1.8 seconds
DB Query Time (95th percentile)	< 100ms	< 75ms
CI/CD Deployment Time	< 15 minutes	~8-10 minutes

Cost Analysis

The project operates well within the ₹5,000/month budget target, primarily leveraging the Azure Student Subscription:

Service	Est. Monthly Cost (₹)
Azure App Service (busbookingapi + busbooking)	Free (Student Credit)
Azure Container Registry (Standard SKU)	~₹400
Azure PostgreSQL Flexible Server	~₹1,200
Bandwidth and Storage	~₹50
Total	~₹1,650

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

Throughout the development of the Bus Booking System project, our team gained hands-on experience across the full software development lifecycle. One key learning was the proper management of Git branches and production environments. We established separate branches for development and production to maintain code integrity.

We developed a strong understanding of containerization and cloud deployment using Docker, Azure Container Registry, and Azure App Service. Configuring Gunicorn as the production WSGI server, setting the correct environment variables, and troubleshooting container startup issues gave us practical exposure to real-world cloud deployment.

Another significant learning came from building and operating a CI/CD pipeline with GitHub Actions. Automating the build, push, and deploy process minimized manual intervention and reduced deployment time significantly. Managing secrets securely via GitHub Actions and injecting them as App Service environment variables reinforced security best practices.

In terms of teamwork, our group followed an industry-standard collaborative approach. Tasks were divided clearly: SIVASABARIGANESAN A led the backend API development and CI/CD pipeline; SREE DEV S handled the database modeling, migrations, and Azure PostgreSQL configuration; SANJAY G managed the frontend development and Azure App Service deployment. Regular discussions, code reviews, and progress tracking kept the team aligned and productive.

CHAPTER 6 - CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

This project successfully delivered a secure, high-performance, college-exclusive bus booking system, fundamentally addressing the lack of a trusted transport platform for hostel students. The platform provides a purpose-built, reliable ecosystem for weekend trip reservations, fulfilling all primary project objectives.

The technical implementation demonstrates mastery of modern cloud-native and DevOps practices. The Django backend, containerized and deployed via Azure App Service with images stored in Azure Container Registry, proved efficient and maintainable. The CI/CD pipeline, automated via GitHub Actions, delivered end-to-end deployments in under 10 minutes with automated database migrations.

From a user and institutional perspective, the platform creates significant, measurable value. It directly improves transport coordination for students, reduces manual booking effort for staff, and creates a high-trust environment through institutional verification and controlled route access.

6.2 FUTURE SCOPE

Short-to-Medium Term: Enhancing the Core Platform

- **Intelligent Product Recommendation:** Develop an advanced recommendation system that suggests products tailored to a student's department, academic year, and common course requirements.
- **Sustainability Rewards Program:** Introduce a gamified rewards program where students earn tokens for listing or purchasing secondhand items, redeemable for discounts or partner offers.
- **Community Lost and Found Hub:** Create a dedicated Lost and Found section allowing students to report and browse lost items with images and descriptions.

Long-Term Vision: Strategic Expansion

- **National Expansion:** Scale beyond a single institution to support a multi-tenant model across multiple colleges, requiring evolution to a multi-instance architecture with centralized identity management.
- **Infrastructure as Code:** Adopt Terraform to manage all Azure resources declaratively, enabling reproducible environment provisioning and simplifying disaster recovery.
- **Advanced Monitoring:** Integrate Azure Application Insights and Grafana for real-time dashboards, distributed tracing, and proactive alerting.

REFERENCES

8. Microsoft Azure. (2025). App Service documentation. <https://learn.microsoft.com/en-us/azure/app-service/>
9. Microsoft Azure. (2025). Azure Database for PostgreSQL Flexible Server documentation. <https://learn.microsoft.com/en-us/azure/postgresql/>
10. Microsoft Azure. (2025). Azure Container Registry documentation. <https://learn.microsoft.com/en-us/azure/container-registry/>
11. GitHub. (2025). GitHub Actions documentation. <https://docs.github.com/en/actions>
12. Django Software Foundation. (2025). Django documentation. <https://docs.djangoproject.com/>
13. Docker Inc. (2025). Docker documentation. <https://docs.docker.com/>
14. Python Software Foundation. (2025). Python documentation. <https://docs.python.org/3/>
15. Beaulieu, A. (2020). Learning SQL, 3rd ed. O'Reilly Media.
16. Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
17. Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.

APPENDICES

Appendix A – Azure Resource Configuration

App Service Environment Variables (busbookingapi)

Variable	Value / Description
DJANGO_SETTINGS_MODULE	transport_booking.settings_production
DATABASE_URL	postgresql://<user>:<pass>@busbookingpg2026.postgres.database.azure.com/b
ALLOWED_HOSTS	* (permits Azure internal probe host)
WEBSITES_PORT	8000 (Gunicorn listen port)
DJANGO_SECRET_KEY	Injected from GitHub Actions secret

Azure Resources Summary

Resource Name	Type	Resource Group	Region
busbookingapi	App Service	busbooking-rg	Central India
busbooking	App Service	busbooking_group	Central India
busbookingcc	Container Registry	busbooking-rg	Central India
busbookingpg2026	PostgreSQL Flexible Server	busbooking-rg	Central India

Appendix B – Request Flow Diagram

High-level request flow:

- Client sends HTTP request to App Service hostname (busbookingapi-fbc8fygubdbmegbz.centralindia-01.azurewebsites.net).
- App Service routes the request to the running Django backend container (pulled from ACR: busbookingcc.azurecr.io/bus-booking-backend:latest).
- Django processes the request using Gunicorn on port 8000.
- Django reads/writes data from PostgreSQL Flexible Server (busbookingpg2026.postgres.database.azure.com) over SSL.
- Response is returned to the client.

Appendix C – CI/CD Pipeline Overview

The GitHub Actions pipeline (main branch) follows these steps:

23. Checkout code from the repository.
24. Authenticate with Azure using AZURE_CREDENTIALS secret.
25. Log in to Azure Container Registry (busbookingcc).
26. Build the Docker image from the backend Dockerfile.
27. Tag the image with both :latest and a versioned tag (vYYYYMMDD-<shortsha>).
28. Push both tags to busbookingcc.azurecr.io.
29. Restart the Azure App Service (busbookingapi) to pull and run the new image.
30. Run Django migrations against the production DATABASE_URL.